

講義 5：Internal Table (內表) 進階

對應練習：ex05 | 答案程式：ZR_TR05_ITAB_ADVANCED

本講重點

- **SORT**：單欄 / 多欄、升冪 / 降冪
- **MODIFY**：改表中既有的列 (搭配 INDEX / TRANSPORTING)
- **DELETE**：依 INDEX、依 WHERE、去重複 (ADJACENT DUPLICATES)
- **INSERT ... INDEX**：插在指定位置
- **DESCRIBE TABLE / lines()** 取筆數
- 這些指令與 **sy-subrc** / **sy-tabix** 的互動

1. SORT：排序

```
SORT gt_students BY score DESCENDING.      " 成績由高到低
SORT gt_students BY score.                  " 不寫就是 ASCENDING
SORT gt_students BY grade ASCENDING
                                     score DESCENDING.      " 多欄：先等第、同等第再比成績
```

- 不加 **BY** 時依表格定義的 KEY 排序——**建議永遠明寫 BY**，意圖清楚。
- 排序會改變 INDEX：排序前記下的「第 n 筆」排序後就不算數了。
- 同值想保持原本相對順序，加 **STABLE**：**SORT gt BY score DESCENDING STABLE.**

排序後可以用二分搜尋加速 READ (大表差很多)：

```
SORT gt_students BY id.
READ TABLE gt_students INTO gs_student WITH KEY id = 'S0002' BINARY SEARCH.
```

BINARY SEARCH 的前提是已依同一組欄位排序，沒排序就用會拿到錯誤結果 (不是報錯！)。

2. MODIFY：修改表中的列

MODIFY 把 work area 的內容寫回表中指定的列：

```
* 指定第幾筆
READ TABLE gt_students INTO gs_student INDEX 2.
gs_student-score = 99.
MODIFY gt_students FROM gs_student INDEX 2.

* LOOP 中可省略 INDEX ( 自動用目前這一筆，即 sy-tabix )
LOOP AT gt_students INTO gs_student.
    gs_student-score = gs_student-score + 5.      " 全班加 5 分
    MODIFY gt_students FROM gs_student.
ENDLOOP.
```

只想更新部分欄位時用 **TRANSPORTING**，其他欄位不動：

```
MODIFY gt_students FROM gs_student INDEX 2 TRANSPORTING score.
```

最常見的坑：LOOP INTO 改了 work area 卻忘了 **MODIFY**——迴圈結束表格完全沒變，程式也不報錯。講義 16 的 **LOOP ... ASSIGNING** 就是為了根治這個問題。

3. DELETE：刪除列

```
DELETE gt_students INDEX 3.           " 刪第 3 筆
DELETE gt_students WHERE score < 60.  " 刪所有不及格 ( 可能多筆 )
```

- **sy-subrc**：有刪到 0，沒刪到 4——照鐵律檢查。
- 去除重複 (先排序再用，只比相鄰列)：

```
SORT gt_students BY id.
DELETE ADJACENT DUPLICATES FROM gt_students COMPARING id.
```

不加 **COMPARING** 則比整列。忘記先 **SORT** 是經典錯誤：不相鄰的重複不會被刪。

注意：LOOP 進行中 **DELETE** 同一張表雖然合法 (刪目前列可用 **DELETE gt_students.** 搭配隱含 index)，但容易寫出難懂的邏輯——初學建議「先 LOOP 蒐集、迴圈外再刪」或直接用 **DELETE ... WHERE**。

4. INSERT：插在指定位置

APPEND 只能加在表尾；要插在中間用 **INSERT**：

```
gs_student-id = 'S0009'. gs_student-name = '插班生'. gs_student-score = 70.
INSERT gs_student INTO gt_students INDEX 2.           " 插成第 2 筆，原本的往後推
```

省略 INDEX 的 **INSERT ... INTO TABLE gt.** 用於 SORTED/HASHED 表 (依 KEY 找位置)，STANDARD 表日常還是 **APPEND** 為主。

5. 取得筆數

```
DATA gv_lines TYPE i.
DESCRIBE TABLE gt_students LINES gv_lines.           " 傳統寫法
WRITE: / '共', gv_lines, '筆'.

gv_lines = lines( gt_students ).                       " 內建函數寫法 ( 較新、較簡潔 )
```

兩種都要看得懂；判斷「表是不是空的」還可以用 **IF gt_students IS INITIAL.**

6. 綜合範例

```

* 需求：去重複 → 全班加 5 分 → 刪不及格 → 依成績排名輸出
SORT gt_students BY id.
DELETE ADJACENT DUPLICATES FROM gt_students COMPARING id.

LOOP AT gt_students INTO gs_student.
    gs_student-score = gs_student-score + 5.
    MODIFY gt_students FROM gs_student TRANSPORTING score.
ENDLOOP.

DELETE gt_students WHERE score < 60.

SORT gt_students BY score DESCENDING.
LOOP AT gt_students INTO gs_student.
    WRITE: / sy-tabix, gs_student-id, gs_student-name, gs_student-score.
ENDLOOP.
WRITE: / '倖存', lines( gt_students ), '人'.

```

7. 常見錯誤與陷阱

症狀	原因
LOOP 裡改分數，結束後表沒變	忘了 MODIFY (INTO 是複本)
BINARY SEARCH 找不到明明存在的資料	沒先依查詢欄位 SORT
ADJACENT DUPLICATES 沒刪乾淨	沒先 SORT，重複列不相鄰
排序後用舊 INDEX 讀到別筆	SORT 之後列的位置全變了
DELETE WHERE 之後筆數不如預期	條件寫反；用 sy-subrc 與 lines() 驗證

8. 補充：舊程式常見的 Header Line 寫法 (為什麼現在不建議用)

翻舊程式 (如本專案的 **ZDQM** 系列) 常看到這種宣告：

```
DATA: itab LIKE ztable OCCURS 0 WITH HEADER LINE.
```

這一行其實疊了兩個各自獨立、各自過時的語法元素，混在一起看容易搞混：

- **OCCURS n**：跟 **WITH HEADER LINE** 完全無關，單獨存在也合法 (`DATA: itab LIKE ztable OCCURS 0.` 不帶 Header Line 一樣能寫)。它是舊時代「預先跟系統要 n 筆資料的記憶體空間」的效能提示，官方文件 (**ABAPDATA_OCCURS**) 明講這行跟現代寫法的 **INITIAL SIZE n** 是同一件事、只是換了關鍵字：`DATA itab TYPE STANDARD TABLE OF ... WITH NON-UNIQUE DEFAULT KEY INITIAL SIZE n.`。官方效能指引 (**ABENINITIAL_MEMORY_REQU_GUIDL**) 進一步說明：現代 **ABAP** 執行環境的自動記憶體配置，對一般 (非巢狀) 內部表已經夠用，不需要手動指定 **INITIAL SIZE**——這個提示現在真正還有意義的場合，只剩「深層表格 (Deep Table，見下一節) 裡的內層小表」這種特殊情境，一般寫法看到的 **OCCURS 0 / OCCURS 10** 這類數字，在今天的系統上基本上只是歷史殘留、不影響行為。

- **WITH HEADER LINE**：這才是真正製造出「同名雙義」問題的部分——宣告一個叫 **itab** 的 internal table，還偷偷生成一個同名的「**work area**」（型別是這張表的 row type），兩者共用同一個識別字 **itab**。LOOP AT **itab**. (不用 INTO) 之所以能動，就是因為每次讀出的那一列會自動塞進這個隱藏 work area。

問題所在：**itab** 這個名字在不同語境 (Operand Position) 下指的到底是「整張表」還是「這一系列的 work area」，規則並不直覺——預設指的是 **work area (Header Line)**，要存取整個表格本體得刻意加中括號 **itab[]** 才行。像 SORT **itab[]** BY ... 這種語句，一旦漏了 **[]**，程式不會報錯，只是動到的是 Header Line 而不是整張表，結果悄悄不對——這正是 SAP 官方文件把 Header Line / TABLES 相關語法列為 obsolete 的核心理由：**同名雙義、要靠符號才能消歧義**，是經典的隱藏 **bug** 來源。

只寫 **OCCURS 0**、不寫 **WITH HEADER LINE**，跟兩者都寫，差別在哪：

	DATA: itab1 LIKE ztable OCCURS 0.	" 只有 OCCURS，沒有 Header Line
	DATA: itab2 LIKE ztable OCCURS 0 WITH HEADER LINE.	" 兩個都有
	itab1 (只有 OCCURS)	itab2 (加了 WITH HEADER LINE)
有沒有隱藏 work area	✗ 沒有，itab1 純粹就是一張表	✓ 有，itab1 之於 itab2 多了一個同名 work area
LOOP AT 寫法	一定要 LOOP AT itab1 INTO ls_xxx. (沒有隱藏 work area 可以塞)	可以省略 INTO，LOOP AT itab2. 直接把每列塞進隱藏的 itab2 work area
itab1 / itab2 這個名字本身有沒有雙義問題	✗ 沒有，就是表格本身，行為跟現代 TYPE STANDARD TABLE OF 宣告的表格一致	✓ 有，預設指 Header Line，要 itab2[] 才能指到表格本體
對應的現代寫法	DATA itab1 TYPE STANDARD TABLE OF ztable WITH NON-UNIQUE DEFAULT KEY INITIAL SIZE 0.	沒有直接對應——現代寫法會拆成「表格 + 獨立 work area」兩個變數，不會做出這種同名雙義的東西

換句話說：**OCCURS n** 本身只是效能提示，不會製造雙義問題；真正的問題來源、也是被官方點名棄用的核心，是 **WITH HEADER LINE**。只寫 **OCCURS 0** (不加 Header Line) 的舊程式，讀起來、行為都跟現代寫法差不多，只是「初始配置筆數」用了舊關鍵字表示，不用特別提防；看到 **WITH HEADER LINE** 才要留意「這個名字可能有雙重身份」。

⚠ 還有一個更該提防的變體：宣告結構的同時直接加 **OCCURS n**，Header Line 這時候強制生成、無法關閉：

DATA: BEGIN OF itab3 OCCURS 0, field1 TYPE i, field2 TYPE string, END OF itab3.	" 這裡不是先定義結構、再變成表格——是「一步到位」
--	----------------------------

這個寫法不是「先宣告結構、再變成 Table Type」兩階段的東西，而是一個陳述式同時做完三件事：定義欄位 (結構) + 宣告成 Internal Table + 生成同名 Header Line。跟前面 **itab1 / itab2** 最大的差別是：**itab1** 那種「LIKE ztable OCCURS n」形式，**WITH HEADER LINE** 是可選的；但 BEGIN OF ... OCCURS n ... END OF 這個形式，官方文件 (ABAPDATA_BEGIN_OF_OCCURS) 明講：

The creation of the header line **cannot be disabled** in this variant. Since header lines in internal tables should never be used, however, **this way of declaring internal tables should never occur again.**

也就是官方對這個特定寫法的評價比一般 Header Line 語法還要更重——沒有辦法只要表格不要 **Header Line**，想避開就只能拆成兩步：先定義結構、再另外宣告一個 **TYPE STANDARD TABLE OF** 的表格（不能用 **BEGIN OF ... OCCURS ... END OF** 這一步到位的寫法）：

```
TYPES: BEGIN OF ty_line,
        field1 TYPE i,
        field2 TYPE string,
    END OF ty_line.
DATA gt_itab TYPE STANDARD TABLE OF ty_line.      " 純表格，沒有 Header Line
```

現代寫法一律「純表格 + 獨立宣告的 work area」，兩個名字不同，意圖清楚，完全沒有這個雙義問題：

```
DATA: gt_itab TYPE STANDARD TABLE OF ztable,
      gs_itab TYPE ztable.

LOOP AT gt_itab INTO gs_itab.
    ...
ENDLOOP.
```

看得懂舊程式在幹嘛就好，自己寫一律用上面這種「itab + 獨立 work area」的寫法——這也是為什麼 **Function Module** 的 **TABLES** 參數（本質就是「帶 Header Line 的內部表」）在 SE37 介面裡也被列為過時語法，細節見講義 15 第 3.1 節。

為什麼 **SAP** 不乾脆把這種語法整個拿掉、逼大家改寫：官方文件對「obsolete」語法的統一定位（**ABENABAP_OBSOLETE**）講得很直白——這些語法「僅為了與舊版本相容而保留（only available for reasons of compatibility with older releases）」，舊程式裡可能還看得到，但新程式不應該再用。SAP 的系統一升級就是幾十年的既有客戶程式碼庫，不能因為語言演進就讓舊程式全部編譯失敗，所以策略是「新寫法優先教、舊寫法保留但不推薦、Class 裡直接禁用倒逼新程式碼走新路」——**WITH HEADER LINE / TABLES** 參數會在 Class / Method 裡完全編譯不過（本檔第 39 節、第 9 節都印證過這點），就是這個策略在語法層級的具體落實：舊寫法留給看得懂 Report/FM 舊碼的人，新的物件導向程式碼從語言設計上直接杜絕重蹈覆轍。

9. 補充：Deep Structure / Deep Table（結構裡包一個 Internal Table）

講義 3 第 6 節學過「巢狀結構」——結構的欄位是另一個結構（1 對 1）。ABAP 還允許結構的欄位是 **Internal Table**（1 對多），這種結構叫 **Deep Structure**（深層結構）；如果表格的每一列本身又是一個 Deep Structure，這種表就叫 **Deep Table**（深層表格）——業界也常用「Nested Table / 巢狀內部表」稱呼同一件事。

典型場景：講義 21 / 23 教的「訂單主檔 + 明細」，之前是用兩張獨立的 DB 表（**ZTR23_ORDH / ZTR23_ORDI**）+ JOIN 做關聯——這是存進資料庫的正規做法。但如果只是要在記憶體裡把「一張訂單的主檔 + 它底下所有明細」當成一個整體處理（例如要傳給一支 FM、或組一份要輸出的 JSON），Deep Structure 更直覺：

```
TYPES: BEGIN OF ty_item,
        posnr TYPE i,
```

```

    matnr TYPE string,
    qty    TYPE i,
END OF ty_item.

```

```

TYPES tt_item TYPE STANDARD TABLE OF ty_item WITH NON-UNIQUE KEY posnr.

```

```

TYPES: BEGIN OF ty_order,          " ← Deep Structure : ITEMS 欄位是一整張 Internal Table
       ordno    TYPE string,
       customer TYPE string,
       items     TYPE tt_item,
END OF ty_order.

```

```

DATA: gs_order TYPE ty_order,
      gs_item  TYPE ty_item.          " 內層 LOOP 用的 work area

```

```

gs_order-ordno    = 'S00001'.
gs_order-customer = 'ACME'.
gs_item-posnr = 10. gs_item-matnr = 'MAT-A'. gs_item-qty = 5.
APPEND gs_item TO gs_order-items.
gs_item-posnr = 20. gs_item-matnr = 'MAT-B'. gs_item-qty = 3.
APPEND gs_item TO gs_order-items.

```

```

LOOP AT gs_order-items INTO gs_item.
    WRITE: / gs_order-ordno, gs_item-posnr, gs_item-matnr, gs_item-qty.
ENDLOOP.

```

補充：行內宣告 (Inline Declaration) 新寫法：7.40 之後，`LOOP AT ... INTO` 的 work area 可以在使用處直接宣告，寫成 `LOOP AT gs_order-items INTO DATA(gs_item)`。——省掉上面 `gs_item TYPE ty_item` 那一行，型別由內表的列自動推導，效果完全相同。本課程前半段一律用「先宣告再使用」的傳統寫法，行內宣告留到講義 26 系統性教；讀到 `INTO DATA(...)` 時，知道它等於「宣告 + LOOP」合在一起即可。

補充：VALUE #(...) 新寫法：7.40 之後，塞資料進內表可以不用先填 work area，直接寫 `APPEND VALUE #( posnr = 10 matnr = 'MAT-A' qty = 5 ) TO gs_order-items`。——一句話同時「建出一列 + APPEND」，# 表示型別由目標 (`gs_order-items` 的列) 自動推導，效果與上面兩行 (填欄位 + APPEND) 完全相同。VALUE 的完整用法留到講義 26。

跟講義 3 巢狀結構的差別：巢狀結構是「欄位裡包一個結構」(1 對 1)；Deep Structure 是「欄位裡包一整張表」(1 對多)——正好對應「訂單主檔對明細是 1 對多」的業務關係，不用拆成兩張表也能表達完整的階層關係。

常見用途：

- 呼叫 BAPI / RFC 時，很多標準介面本來就是 Deep Structure (Header 參數裡帶一個 Item 表)
- 組 JSON / XML (REST 課程會用到) 時，巢狀資料天生就是 Deep Structure
- 只是暫時在記憶體整理階層式資料、不需要馬上落地到資料庫時

限制與踩坑：

- Deep Structure / Deep Table 不能用在 **FM** 的 **TABLES** 參數 (見上一節、講義 15 第 3.1 節) ——**TABLES** 只認 flat line type 的 Standard Table · Method 或新式 FM 介面 (**CHANGING/IMPORTING/EXPORTING**) 才能傳遞 Deep Structure · 這也是官方把 **TABLES** 列為 obsolete 的原因之一
- **MOVE-CORRESPONDING** (講義 3) 搬到深層欄位時，預設不會自動遞迴搬內層表格的內容，要加 **EXPANDING NESTED TABLES** 才會連內層表一起搬
- Deep Structure 不能直接拿去 **INSERT/UPDATE** 資料庫表——DB 表本質上是 flat 的，要嘛拆開分別寫兩張表 (講義 23 的做法)，要嘛用支援巢狀資料的 CDS/RAP 技術 (進階課題，本課不涉及)

10. 課堂練習

完成 **ex05**：對學生表做 SORT、MODIFY 加分、DELETE 篩選與去重複，觀察每步的筆數與 sy-subrc。

接下來 (依授課順序) 先上 **lec19 除錯 Debugger** (拿目前的技能實戰抓 bug)，再上 **lec16 Field-Symbol**：解決本講「INTO 複本 + MODIFY」的效能與遺漏問題。